LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

07

Linux Networking

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 Understanding Linux Networking
02 OSI and TCP/IP Models
03 IP Addresses
04 Network Interfaces
05 Subnets and CIDR
06 Gateway and Routing
07 DNS
08 TCP and UDP
09 Ports
10 Network Sockets
11 Opening a Website: Step by Step
12 Troubleshooting Commands
13 Linux Firewalls
14 SSH Remote Access
15 Networking in AWS
16 Troubleshooting Scenarios
17 Interview Q&A
18 Takeaways and Exercises

Ravi Tomar

Linux · DevOps · Cloud Infrastructure

CHAPTER OBJECTIVE

Understand how networking works in Linux, including IP addresses, network interfaces, routing, DNS, ports, sockets and firewalls. Learn practical networking commands and troubleshooting techniques used by DevOps engineers to diagnose connectivity issues between servers, applications and external services.

1 Understanding Linux Networking

Networking allows Linux servers to communicate with other systems over local networks or the internet. For example, consider a typical web application architecture:

```
User Browser
     |
     v
Internet
     |
     v
AWS Load Balancer
     |
     v
Linux EC2 Server
     |
     v
Application (Port 3000)
     |
     v
Database Server (Port 5432)
```

Each component communicates using network protocols, IP addresses and ports. As a DevOps engineer, you should understand:

- IP addressing.
- Network interfaces.
- Subnets and gateways.
- DNS resolution.
- Routing.
- TCP and UDP.
- Ports and sockets.
- Firewall rules.
- Network troubleshooting.

2 The OSI and TCP/IP Models

Network models divide communication into layers. Each layer has a specific job and relies on the layer below it. Interviewers often use these models to check whether you can troubleshoot systematically.

OSI Model (7 Layers)

Layer	Name	Purpose	Examples
7	Application	Network services used by applications	HTTP, DNS, SSH
6	Presentation	Data format and encryption	TLS/SSL, encoding
5	Session	Opening and managing sessions	Session setup
4	Transport	End-to-end delivery using ports	TCP, UDP
3	Network	Logical addressing and routing	IP, ICMP, routers
2	Data Link	Local delivery using MAC addresses	Ethernet, ARP, switches
1	Physical	Transmitting raw bits	Cables, NICs, Wi-Fi

TCP/IP Model (4 Layers)

The TCP/IP model is the practical model used by the internet. It groups the OSI layers:

TCP/IP layer	OSI layers	Examples
Application	5, 6, 7	HTTP, DNS, SSH, TLS
Transport	4	TCP, UDP
Internet	3	IP, ICMP
Network Access	1, 2	Ethernet, Wi-Fi

Troubleshooting Layer by Layer

Question	Layer	Tool
Is the interface up?	1–2	`ip link`
Do I have an IP address and a route?	3	`ip a`, `ip route`, `ping`
Is the port reachable?	4	`nc -zv`, `ss`
Is the application responding correctly?	7	`curl -v`

> **INTERVIEW TIP**
>
> When asked to troubleshoot connectivity, work bottom-up through the layers. It shows a structured approach instead of random command dumping.

3 Understanding IP Addresses

An IP address identifies a device or network interface on a network. There are two major IP versions: IPv4 and IPv6.

IPv4

IPv4 addresses use 32 bits and are commonly written in four decimal sections. Each section ranges from 0 to 255.

```
192.168.1.10
```

Common private IPv4 ranges:

Range	CIDR
10.0.0.0 – 10.255.255.255	10.0.0.0/8
172.16.0.0 – 172.31.255.255	172.16.0.0/12
192.168.0.0 – 192.168.255.255	192.168.0.0/16

Private IP addresses are commonly used inside company networks and cloud VPCs.

IPv6

IPv6 uses 128 bits and is designed to provide a much larger address space.

```
2001:db8::10
```

You may encounter IPv6 addresses when working with modern cloud infrastructure and dual-stack networks.

Public vs Private IP

Public IP	Private IP
Routable over the public internet	Used within private networks
Must be globally unique	Can be reused in different private networks
Often used for internet-facing services	Commonly used for internal communication

4 Understanding Network Interfaces

A network interface connects a Linux system to a network. Examples include `eth0`, `ens5`, `enp0s3` and `lo`. The exact interface name depends on the Linux distribution and hardware or virtual environment.

```
# View network interfaces (also shows MAC addresses)
ip link show

# View IP addresses
ip addr show

# Short form
ip a
```

Example:

```
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP>
    inet 10.0.1.25/24
```

This indicates:

- Interface: `ens5`
- Interface status: UP
- IPv4 address: `10.0.1.25`
- Network prefix: `/24`

```
# Check interface statistics, including errors and dropped packets
ip -s link show ens5

# Show the ARP/neighbour table (IP-to-MAC mappings on the local network)
ip neigh
```

Loopback Interface

The loopback interface is named `lo`. Its common IPv4 address is `127.0.0.1`. It allows a system to communicate with itself.

```
ping 127.0.0.1
```

Applications often use `localhost` or `127.0.0.1` for local communication.

Hostname

```
# Show the hostname
hostnamectl

# Change the hostname permanently
sudo hostnamectl set-hostname web01
```

Persistent Network Configuration

Changes made using the `ip` command are temporary and are lost after a reboot. Permanent configuration depends on the distribution:

Distribution	Tool	Configuration
Ubuntu	Netplan	Files in `/etc/netplan/*.yaml`, applied using `sudo netplan apply`
RHEL-based	NetworkManager	Managed using `nmcli`, for example `nmcli connection show`

5 Understanding Subnets and CIDR

A subnet divides a network into smaller logical networks. CIDR (Classless Inter-Domain Routing) notation represents the network prefix length.

```
192.168.1.10/24
```

Here:

- IP address: `192.168.1.10`
- Prefix: `/24`
- Subnet mask: `255.255.255.0`
- Network address: `192.168.1.0`

A `/24` IPv4 network contains 256 total addresses, of which 254 are traditionally usable host addresses on a conventional subnet.

Quick formula: total addresses = $2^{(32 - \text{prefix})}$. For example, a `/26` has 2^6 = 64 addresses.

Common CIDR Prefixes

CIDR	Subnet mask	Total IPv4 addresses
/8	255.0.0.0	16,777,216
/16	255.255.0.0	65,536
/24	255.255.255.0	256
/25	255.255.255.128	128
/26	255.255.255.192	64
/30	255.255.255.252	4

AWS INTERVIEW NOTE

AWS reserves 5 IP addresses in every subnet (the first four and the last one). A `/24` subnet in AWS therefore has **251** usable addresses, not 254.

DEVOPS NOTE

CIDR knowledge is important when configuring AWS VPCs, subnets, route tables, security groups and Kubernetes networking.

6 Understanding Default Gateway and Routing

A default gateway is the router a system uses when it needs to communicate with destinations outside its directly connected networks.

```
Linux Server: 192.168.1.10
Gateway:      192.168.1.1
```

If the server needs to communicate with an external IP address, it generally sends the traffic to its default gateway.

```
# View routing table
ip route
```

Example:

```
default via 192.168.1.1 dev ens5
192.168.1.0/24 dev ens5 proto kernel scope link
```

Meaning:

- `default via` : Default gateway.
- `192.168.1.0/24` : Directly connected network.
- `dev ens5` : Network interface used.

```
# Check which interface and gateway Linux will use to reach a destination
ip route get 8.8.8.8

# Add a temporary route
sudo ip route add 10.10.0.0/16 via 192.168.1.1

# Delete a route
sudo ip route del 10.10.0.0/16
```

These commands modify runtime routing configuration. Persistent network configuration depends on the distribution and network management system.

7 Understanding DNS

DNS (Domain Name System) translates domain names into IP addresses. Without DNS, users and applications would need to connect using IP addresses instead of domain names.

```
google.com  →  IP address
```

Check DNS Resolution

```
nslookup google.com
dig google.com

# Install dig if missing (Ubuntu)
sudo apt install dnsutils

# Install dig if missing (RPM-based)
sudo dnf install bind-utils

# Check resolution through the system's name service configuration
getent hosts google.com
```

How Linux Resolves a Name

```
Application asks for "api.example.com"
        |
        v
/etc/nsswitch.conf decides the order  (hosts: files dns)
        |
        v
files → /etc/hosts  (local mappings checked first)
        |
        v
dns   → nameserver in /etc/resolv.conf
        |
        v
Recursive resolver asks: root → .com → example.com servers
        |
        v
Answer returned and cached for its TTL
```

INTERVIEW TRAP

`dig` and `nslookup` query DNS servers directly and **ignore** `/etc/hosts`. `getent hosts`, `ping` and most applications follow `/etc/nsswitch.conf`, so they do use `/etc/hosts`. If `dig` and your application disagree about an IP, check `/etc/hosts`.

Common DNS Record Types

Record	Purpose
A	Maps a name to an IPv4 address
AAAA	Maps a name to an IPv6 address
CNAME	Alias that points one name to another name
MX	Mail server for the domain
TXT	Text data, often used for domain verification and email security
NS	Name servers responsible for the domain
PTR	Reverse lookup: maps an IP address back to a name

Useful dig Commands

```
# Show only the answer
dig +short example.com

# Query a specific record type
dig example.com MX

# Query a specific DNS server
dig @8.8.8.8 example.com

# Reverse lookup
dig -x 8.8.8.8
```

DNS answers are cached for their **TTL** (time to live). This is why DNS changes can take time to appear everywhere.

DNS mainly uses **UDP port 53**. It also uses TCP port 53 for large responses and zone transfers.

DNS Configuration

```
# Check resolver configuration
cat /etc/resolv.conf
```

Example:

```
nameserver 8.8.8.8
nameserver 1.1.1.1
```

These are public DNS resolver addresses.

On Ubuntu, `/etc/resolv.conf` usually shows `nameserver 127.0.0.53`. This is the local systemd-resolved cache, not the real upstream server. Check the real DNS servers using:

```
resolvectl status
```

Understanding /etc/hosts

The `/etc/hosts` file provides local hostname-to-IP mappings.

```
192.168.1.20  appserver.local
192.168.1.30  dbserver.local
```

```
cat /etc/hosts
```

The hosts file can override DNS resolution for matching names, depending on the system's name-service configuration.

> **DEVOPS SCENARIO**
>
> If an application cannot connect to a database using its hostname, check DNS resolution, `/etc/hosts` and the configured resolver before investigating the application itself.

8 Understanding TCP and UDP

TCP and UDP are two important transport-layer protocols.

TCP (Transmission Control Protocol)

TCP provides reliable, connection-oriented communication.

- Establishes a connection before transferring data.
- Ensures data arrives in order.
- Retransmits lost data.
- Provides flow and congestion control.

Common uses: HTTP and HTTPS, SSH, database connections, file transfers.

UDP (User Datagram Protocol)

UDP is connectionless and does not guarantee delivery or ordering.

- Lower protocol overhead.
- No built-in retransmission.
- No connection establishment.
- Useful for latency-sensitive communication.

Common uses: DNS queries, DHCP, some streaming and real-time communication, certain monitoring and telemetry protocols.

TCP vs UDP

TCP	UDP
Connection-oriented	Connectionless
Reliable delivery	No delivery guarantee
Ordered data	No ordering guarantee
More protocol overhead	Lower protocol overhead
SSH, HTTPS	DNS, DHCP

TCP Three-Way Handshake

Before any data is sent, TCP sets up a connection in three steps:

```
Client                          Server
  |  ------- SYN ---------->     |   "I want to connect"
  |  <------ SYN-ACK -------     |   "OK, I am ready"
  |  ------- ACK ---------->     |   "Confirmed"
  |                              |
  |   Connection established     |
```

Closing a connection uses a similar exchange of **FIN** and **ACK** packets from both sides.

Important TCP Connection States

State	Meaning
LISTEN	Server is waiting for incoming connections
SYN-SENT	Client sent SYN and is waiting for a reply
ESTABLISHED	Connection is open and data can flow
TIME-WAIT	The side that closed first waits briefly so delayed packets are handled safely
CLOSE-WAIT	The remote side closed, but the local application has not closed its socket yet

```
# Count connections by state
ss -tan | awk 'NR>1 {print $1}' | sort | uniq -c

# Summary of all sockets
ss -s
```

INTERVIEW INSIGHT

Many **TIME-WAIT** connections are normal on busy servers. A growing number of **CLOSE-WAIT** connections usually means the application is not closing its sockets properly, which is an application bug.

9 Understanding Ports

A port identifies a particular application or service on a server. For example, one Linux server can run Nginx, SSH and a database simultaneously using different ports.

Important Ports for DevOps Engineers

Port	Service	Purpose
22	SSH	Remote server access
25	SMTP	Email transfer
53	DNS	Domain name resolution
80	HTTP	Web traffic
123	NTP	Time synchronization
443	HTTPS	Secure web traffic
3306	MySQL	Database
5432	PostgreSQL	Database
6379	Redis	In-memory data store
8080	HTTP alternate	Common application port
3000	Node.js apps	Common development/application port
9090	Prometheus	Common metrics server port

Ports are not permanently tied to a particular application. Administrators can configure services to listen on different ports.

Port Ranges

Range	Name	Use
0 – 1023	Well-known (privileged)	Standard services; binding requires root privileges
1024 – 49151	Registered	Applications such as databases
49152 – 65535	Dynamic / ephemeral	Temporary client-side ports

Linux uses its own ephemeral range for outgoing connections. Check it using `cat /proc/sys/net/ipv4/ip_local_port_range`.

INTERVIEW TRAP QUESTION

Why can't an application running as a normal user listen on port 80?

Ports below 1024 are privileged. Common solutions are: run the application on a high port (such as 3000) behind a reverse proxy like Nginx on port 80, grant the binary the `cap_net_bind_service` capability, or set `AmbientCapabilities=CAP_NET_BIND_SERVICE` in its systemd unit. Running the application as root is not recommended.

Check Listening Ports

```
sudo ss -tulnp
```

Option	Meaning
`t`	TCP sockets
`u`	UDP sockets
`l`	Listening sockets
`n`	Numeric addresses and ports
`p`	Show process information

```
# Check a particular port
sudo ss -lntp | grep :80

# Check whether a process is listening on port 3000
sudo ss -lntp | grep :3000

# Exact port match (grep :80 also matches :8080)
sudo ss -lntp 'sport = :80'
```

netstat

An older networking command. It may require installation.

```
sudo apt install net-tools
sudo netstat -tulnp
```

Prefer `ss` on modern Linux systems.

10 Understanding Network Sockets

A socket is an endpoint used by applications for network communication. For example, an application listening on `0.0.0.0:3000` accepts connections on port 3000 through all available IPv4 interfaces.

Address	Meaning
`127.0.0.1`	Accept connections only through IPv4 loopback
`0.0.0.0`	Listen on all IPv4 interfaces
`::1`	IPv6 loopback
`::`	IPv6 wildcard address

> **IMPORTANT**
>
> Listening on `0.0.0.0` does not automatically make an application reachable from the internet. Firewalls, routing, cloud security groups and other network controls still apply.

Practical Example

Suppose a Node.js application listens on port 3000.

- If it binds only to `127.0.0.1:3000`, it is reachable locally but normally not directly reachable from another machine.
- If it binds to `0.0.0.0:3000`, it can accept connections through its IPv4 network interfaces, subject to network access controls.

11 What Happens When You Open a Website?

"What happens when you type a URL into a browser?" is one of the most common networking interview questions. It connects almost every topic in this chapter.

Example: you open `https://example.com`.

Step 1 DNS resolution

The browser checks its cache, then the operating system resolves `example.com` to an IP address using `/etc/hosts` and DNS.

Step 2 TCP connection

The browser opens a TCP connection to that IP on port 443 using the three-way handshake (SYN, SYN-ACK, ACK).

Step 3 TLS handshake

The server sends its certificate. The browser verifies that it is valid, trusted and matches the domain. Both sides agree on encryption keys, creating a secure channel.

Step 4 HTTP request

The browser sends an HTTP request, such as `GET /`, over the encrypted connection.

Step 5 Server-side processing

The request typically passes through a load balancer to a web server, which forwards it to the application. The application may query a database before building a response.

Step 6 HTTP response

The server returns a response with a status code (such as `200 OK`) and the page content.

Step 7 Rendering

The browser renders the page and requests additional resources such as CSS, JavaScript and images, often reusing the same connection.

> **INTERVIEW TIP**
>
> Mention the tool that tests each step: `dig` for DNS, `nc -zv` for TCP, `openssl s_client` for TLS and `curl -v` for the full request. This shows you can turn theory into troubleshooting.

12 Network Troubleshooting Commands

These commands are especially important for DevOps interviews and production troubleshooting.

ping

Tests basic IP connectivity using ICMP echo requests.

```
ping -c 4 8.8.8.8
ping -c 4 google.com
```

If the IP works but the hostname fails, investigate DNS.

> **IMPORTANT**
>
> Some systems block ICMP. A failed ping does not always mean the application or server is unreachable.

traceroute, tracepath and mtr

Show the network path towards a destination.

```
traceroute google.com

# Install if required
sudo apt install traceroute

# Alternative
tracepath google.com

# mtr combines ping and traceroute, showing packet loss per hop
mtr -rw google.com
```

Some intermediate routers may not respond to traceroute probes, so missing hops do not necessarily indicate a failure.

curl

Tests HTTP and HTTPS endpoints.

```
curl https://example.com

# Check response headers
curl -I https://example.com

# Test a local application
curl http://localhost:3000

# Show detailed connection information
curl -v https://example.com

# Print only the HTTP status code
curl -s -o /dev/null -w "%{http_code}\n" https://example.com

# Measure where time is spent
curl -s -o /dev/null -w "dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appcon

# Send a request for a domain to one specific backend server
curl --resolve example.com:443:10.0.1.25 https://example.com
```

HTTP Status Codes

Code	Meaning
200	OK: request succeeded
301 / 302	Redirect to another URL
400	Bad request from the client
401	Unauthorized: authentication required or failed
403	Forbidden: authenticated or not, access is not allowed
404	Not found
500	Internal server error inside the application
502	Bad Gateway: the proxy or load balancer got no valid response from the backend (backend down or crashed)
503	Service Unavailable: overloaded, under maintenance, or no healthy backends
504	Gateway Timeout: the backend took too long to respond

wget

Downloads files from HTTP, HTTPS or FTP sources.

```
wget https://example.com/file.zip
```

nc (Netcat)

Useful for testing TCP connectivity to a particular port.

```
nc -zv 192.168.1.20 5432
```

This checks whether a TCP connection can be established to port 5432.

openssl (TLS Certificates)

```
# Inspect the TLS connection and certificate
openssl s_client -connect example.com:443 -servername example.com

# Show certificate validity dates and subject
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null | op
```

tcpdump

Captures and analyzes network packets. The `-nn` option shows raw IPs and port numbers instead of names.

```
# Capture traffic on a particular interface
sudo tcpdump -i eth0

# Capture traffic on port 443
sudo tcpdump -nn -i eth0 port 443

# Capture traffic involving a particular host
sudo tcpdump -nn -i eth0 host 192.168.1.20

# Save captured packets to a file
sudo tcpdump -i eth0 -w capture.pcap
```

> **WARNING**
>
> Packet captures can contain sensitive information. Capture only what is necessary and protect the resulting files.

13 Understanding Linux Firewalls

A firewall controls network traffic based on configured rules. Linux commonly uses Netfilter as its packet-filtering framework. Tools used to manage firewall rules include UFW, firewalld, nftables and iptables.

UFW (Uncomplicated Firewall)

UFW is commonly used on Ubuntu.

```
# Check status
sudo ufw status

# Allow SSH first, before enabling the firewall
sudo ufw allow 22/tcp

# Enable firewall
sudo ufw enable

# Allow HTTP, HTTPS and an application port
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow 3000/tcp

# Show rules with numbers
sudo ufw status numbered

# Delete a rule
sudo ufw delete allow 3000/tcp
```

WARNING

Before enabling a firewall on a remote server, ensure SSH access is allowed. Otherwise, you may lock yourself out.

firewalld

Commonly used on RHEL-based distributions.

```
# Check status
sudo firewall-cmd --state

# List active rules
sudo firewall-cmd --list-all

# Show active zones
sudo firewall-cmd --get-active-zones

# Allow HTTP permanently
sudo firewall-cmd --permanent --add-service=http

# Allow a custom TCP port
sudo firewall-cmd --permanent --add-port=3000/tcp

# Reload firewall rules
sudo firewall-cmd --reload
```

INTERVIEW TRAP

Without `--permanent`, a firewalld rule applies immediately but is lost after a reload or reboot. With `--permanent`, the rule is saved but takes effect only after `--reload`.

nftables and iptables

`nftables` is the modern Linux packet-filtering framework. `iptables` is an older interface that may still be present on many systems, sometimes implemented through an nftables compatibility layer.

```
# Inspect nftables rules
sudo nft list ruleset

# Inspect iptables rules
sudo iptables -L -n -v
```

iptables organizes rules into chains:

Chain	Applies to
INPUT	Traffic coming into this server
OUTPUT	Traffic leaving this server
FORWARD	Traffic passing through this server to another host

```
# Example: allow incoming HTTP traffic
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

Rules are checked from top to bottom, and the **first matching rule wins**. iptables rules are not saved across reboots unless you save them, for example with `iptables-save` or the `iptables-persistent` package.

> **IMPORTANT**
>
> Do not assume that every Linux server uses UFW or firewalld. Identify the firewall system before changing rules.

14 SSH – Remote Linux Access

SSH (Secure Shell) allows secure remote access to Linux servers.

```
# Connect to a server
ssh username@192.168.1.20

# Connect using a custom port
ssh -p 2222 username@192.168.1.20

# Connect using a private key
ssh -i mykey.pem username@192.168.1.20

# Copy a file to a remote server
scp app.zip username@192.168.1.20:/opt/myapp/

# Copy a file from a remote server
scp username@192.168.1.20:/var/log/app.log .

# Sync a directory efficiently (copies only changes)
rsync -avz app/ username@192.168.1.20:/opt/myapp/
```

Key-Based Authentication

```
# 1. Generate a key pair on your machine
ssh-keygen -t ed25519 -C "ravi@laptop"

# 2. Copy the public key to the server
ssh-copy-id username@192.168.1.20
```

The public key is added to `~/.ssh/authorized_keys` on the server. The private key never leaves your machine.

> **INTERVIEW TRAP**
> SSH refuses key login if permissions are too open. On the server, `~/.ssh` must be `700` and `authorized_keys` must be `600`. On your machine, a private key such as `mykey.pem` must be `400` or `600`, otherwise SSH shows *"UNPROTECTED PRIVATE KEY FILE"*.

SSH Client Configuration

Save connection details in `~/.ssh/config` so you can connect with a short name:

```
Host web01
    HostName 10.0.1.25
    User ubuntu
    IdentityFile ~/.ssh/mykey.pem
```

```
ssh web01
```

Bastion (Jump) Host

Servers in private subnets are often reached through a bastion host in a public subnet:

```
ssh -J ubuntu@bastion-public-ip ubuntu@10.0.2.15
```

Debugging SSH

```
# Verbose output shows exactly where the connection fails
ssh -v username@192.168.1.20
```

SSH Server Configuration and Hardening

The server-side SSH configuration is commonly stored in `/etc/ssh/sshd_config`. Common hardening settings:

```
PermitRootLogin no
PasswordAuthentication no
AllowUsers deploy ravi
```

```
# Test the configuration before restarting
sudo sshd -t
```

> **WARNING**
>
> Keep your current SSH session open while restarting the SSH service. If the new configuration is wrong, you can still fix it from the open session instead of being locked out.

The SSH service is commonly named `ssh` on Ubuntu and `sshd` on many RHEL-based distributions.

```
# Ubuntu
sudo systemctl status ssh

# RHEL-based
sudo systemctl status sshd
```

> **DEVOPS NOTE**
>
> SSH is essential for administering EC2 instances, on-premises Linux servers and remote deployment environments. Key-based authentication is commonly preferred for production access.

15 Linux Networking in AWS

When Linux servers run on AWS, several cloud network layers sit around the operating system.

Component	Purpose
VPC	Your private network in AWS, defined by a CIDR block
Subnet	A range of IPs inside a VPC, in one Availability Zone
Internet Gateway	Allows resources in public subnets to reach the internet
NAT Gateway	Allows private subnet instances to make outbound internet connections only
Route table	Decides where traffic from a subnet is sent
Security Group	Firewall attached to an instance's network interface
Network ACL	Firewall applied at the subnet level
Elastic IP	A static public IPv4 address

A subnet is **public** when its route table sends `0.0.0.0/0` to an Internet Gateway. Otherwise it is **private**.

Security Group vs Network ACL

Feature	Security Group	Network ACL
Applies to	Instance (network interface)	Subnet
State	Stateful: return traffic is allowed automatically	Stateless: return traffic must be allowed explicitly, including ephemeral ports
Rules	Allow rules only	Allow and deny rules
Evaluation	All rules are evaluated	Rules are processed in number order

Instance Metadata

An EC2 instance can query information about itself from `169.254.169.254`. Modern instances use IMDSv2, which requires a session token:

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/local-ipv4
```

Checklist: EC2 Application Not Reachable

1. Application running and listening on the right address and port (`ss -lntp`).
2. Responds locally (`curl localhost:PORT`).
3. Linux firewall allows the port (UFW, firewalld, iptables).
4. Security Group allows the port from the right source.
5. Network ACL allows the traffic in both directions.
6. Route table has the correct route (Internet Gateway or NAT Gateway).
7. Instance has a public IP or Elastic IP, if accessed directly from the internet.
8. If behind a load balancer, targets are healthy.

16 Practical DevOps Networking Scenarios

Scenario 1 Website is not accessible

Suppose a Node.js application runs on port 3000.

```
# Check whether the process is running
ps aux | grep node

# Check whether port 3000 is listening
sudo ss -lntp | grep :3000

# Test locally
curl -v http://localhost:3000

# Check firewall rules
sudo ufw status
```

If running on AWS, also verify the EC2 security group, network ACLs, route tables and load balancer configuration, using the checklist in Section 15.

Scenario 2 Server cannot resolve a domain name

```
# Test IP connectivity
ping -c 4 8.8.8.8

# Test DNS
getent hosts google.com

# Inspect resolver configuration
cat /etc/resolv.conf

# Query DNS directly
dig google.com
```

This helps separate DNS problems from general network connectivity problems.

Scenario 3 Application cannot connect to a database

Suppose the application server needs to connect to PostgreSQL at `10.0.2.20:5432`.

```
# Test connectivity from the application server
nc -zv 10.0.2.20 5432

# Check the database server's listening port
sudo ss -lntp | grep :5432
```

Investigate:

- Database service status.
- Database bind address.
- Server firewall rules.
- AWS security groups.
- Network ACLs and routing.
- Database authentication and application configuration.

A successful TCP connection does not guarantee that database authentication or queries will work.

Scenario 4 Port is already in use

```
Address already in use
```

```
# Find the process using port 3000
sudo ss -lntp | grep :3000

# Alternatively
sudo lsof -i :3000
```

Identify the process before stopping it or changing the application's port.

Scenario 5 Application works locally but not from another server

```
sudo ss -lntp
```

If it listens only on `127.0.0.1`, remote systems cannot normally connect directly. Check:

- Application bind address.
- Linux firewall.
- Cloud security groups.
- Network routes.
- Destination server's listening port.

Scenario 6 Users see an SSL certificate error

```
# Check the certificate's validity dates
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null
```

If `notAfter` is in the past, the certificate has expired. Renew it (for example, `sudo certbot renew` for Let's Encrypt), then reload the web server with `sudo systemctl reload nginx`.

Also check that the certificate matches the domain name and that the full certificate chain is installed. Set up monitoring to alert well before certificates expire.

Scenario 7 dig shows the correct IP, but the application connects to a different one

```
# What the system resolver returns (uses /etc/hosts)
getent hosts api.example.com

# Look for an override
grep api.example.com /etc/hosts
```

`dig` ignores `/etc/hosts`, but applications do not. An old entry in `/etc/hosts` overrides DNS. Also check `/etc/nsswitch.conf`, and remember that some runtimes (such as the JVM) cache DNS results inside the application.

17 Common Interview Questions

Q1. What is an IP address?

An IP address identifies a network interface and allows devices to communicate over an IP network.

Q2. What is the difference between public and private IP addresses?

Public IPs are globally routable over the internet. Private IPs are used within private networks and are not directly routable over the public internet.

Q3. What is a default gateway?

A default gateway is the router used to forward traffic to destinations outside the local network.

Q4. What is DNS?

DNS translates domain names into IP addresses and supports other types of name-related lookups.

Q5. What is the difference between TCP and UDP?

TCP provides connection-oriented, reliable and ordered delivery. UDP is connectionless and does not guarantee delivery or ordering.

Q6. What is a port?

A port identifies a network service or application endpoint on a system.

Q7. How do you check listening ports in Linux?

```
sudo ss -tulnp
```

Q8. What is the difference between `curl` and `ping`?

`ping` tests ICMP connectivity. `curl` makes application-level requests such as HTTP and HTTPS requests.

Q9. What is the difference between `ss` and `netstat`?

Both display network connection and socket information. `ss` is the modern tool and is generally preferred.

Q10. How do you check DNS resolution?

```
nslookup google.com
dig google.com
getent hosts google.com
```

Q11. What is the difference between `127.0.0.1` and `0.0.0.0`?

`127.0.0.1` is the IPv4 loopback address. `0.0.0.0` is the IPv4 wildcard address used by servers to listen on all local IPv4 interfaces.

Q12. What is a firewall?

A firewall controls network traffic according to configured rules.

Q13. How do you troubleshoot a port connectivity issue?

Check whether the destination service is running and listening, test connectivity using `nc`, and investigate host firewall, routing and cloud network controls.

Q14. What is SSH?

SSH is a secure protocol used for remote command execution, administration and file transfer.

Q15. What is CIDR?

CIDR represents IP network prefixes using notation such as `192.168.1.0/24`.

Q16. Why can an application work on localhost but fail remotely?

It may be listening only on the loopback interface, or remote traffic may be blocked by a firewall, security group, routing rule or network ACL.

Q17. Explain the TCP three-way handshake.

The client sends SYN, the server replies with SYN-ACK, and the client confirms with ACK. After these three steps the connection is established and data can be sent.

Q18. What happens when you type a URL into a browser?

The name is resolved to an IP using DNS, a TCP connection is opened with the three-way handshake, a TLS handshake verifies the certificate and sets up encryption, the browser sends an HTTP request, the server processes it and returns a response with a status code, and the browser renders the page.

Q19. What is the difference between a Layer 4 and a Layer 7 load balancer?

A Layer 4 load balancer (such as AWS NLB) routes traffic using IP addresses and ports only. A Layer 7 load balancer (such as AWS ALB) understands HTTP, so it can route based on paths, hostnames and headers.

Q20. What is the difference between HTTP 502 and 504?

502 Bad Gateway means the proxy or load balancer received no valid response from the backend, usually because it is down or crashed. 504 Gateway Timeout means the backend is reachable but took too long to respond.

Q21. What is the difference between a Security Group and a Network ACL?

A Security Group is a stateful firewall at the instance level with allow rules only. A Network ACL is a stateless firewall at the subnet level with allow and deny rules, so return traffic must be allowed explicitly.

Q22. Why might `dig` and `ping` show different IP addresses for the same name?

`dig` queries DNS directly and ignores `/etc/hosts`, while `ping` follows `/etc/nsswitch.conf`, which usually checks `/etc/hosts` first. An entry in `/etc/hosts` explains the difference.

Q23. A server has thousands of connections in CLOSE-WAIT. What does it mean?

The remote side has closed the connections, but the local application has not closed its sockets. This usually indicates a socket leak in the application. Restarting helps temporarily, but the application code must be fixed.

Q24. Why can't a non-root application listen on port 80?

Ports below 1024 are privileged. Run the application on a high port behind a reverse proxy, or grant the `CAP_NET_BIND_SERVICE` capability instead of running it as root.

18 Chapter Takeaways

After completing this chapter, you should understand:

- ✓ The OSI and TCP/IP models and layer-by-layer troubleshooting.
- ✓ IPv4 and IPv6 addressing.
- ✓ Public and private IP addresses.
- ✓ Network interfaces, loopback and persistent network configuration.
- ✓ Subnets and CIDR notation.
- ✓ Default gateways and routing tables.
- ✓ DNS resolution order, record types and `/etc/hosts`.
- ✓ TCP and UDP, the three-way handshake and TCP connection states.
- ✓ Common network ports and privileged ports.
- ✓ Network sockets and listening addresses.
- ✓ What happens when you open a website.
- ✓ Network troubleshooting using `ping`, `curl`, `nc`, `dig`, `openssl` and `tcpdump`.
- ✓ HTTP status codes, including 502, 503 and 504.
- ✓ Linux firewall tools: UFW, firewalld, nftables and iptables.
- ✓ SSH remote administration, key-based authentication and bastion hosts.
- ✓ AWS networking: VPC, subnets, Security Groups and Network ACLs.
- ✓ Practical connectivity troubleshooting.

PRACTICAL EXERCISES

1. Check your Linux IP address using `ip a`.
2. Identify your network interfaces using `ip link`.
3. Find your default gateway using `ip route`.
4. Check DNS resolution using `dig` or `getent`.
5. Test connectivity to a public IP using `ping`.
6. Check listening ports using `ss -tulnp`.
7. Install Nginx and verify that port 80 is listening.
8. Test an HTTP endpoint using `curl -I`.
9. Check whether a remote TCP port is reachable using `nc`.
10. Use `traceroute` or `tracepath` to inspect a network path.
11. Inspect your firewall rules.
12. Capture traffic on a test port using `tcpdump`.
13. Use `curl -w` to measure DNS, connect and total time for a website.
14. Check a website's SSL certificate expiry date using `openssl`.
15. Count TCP connections by state using `ss -tan`.

NEXT CHAPTER

Chapter 08: Linux Boot Process and System Initialization

You will learn what happens when a Linux server starts, how BIOS/UEFI, GRUB, the kernel and systemd work together, how boot targets operate, and how to troubleshoot boot failures.

END OF CHAPTER 7